Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 12–21

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

An Enhanced Dynamic Multi-Path Routing Algorithm for Software Defined Networks

Venkata Sai R.[a], Prabu U.[a,*], Satwik Ch.[a], Sanjaya Kumar Panda[b], Geetha V.[c]

[a]*Department of CSE, Velagapudi Ramakrishna Siddhartha Engineering College, Vijayawada-520007, India*
[b]*Department of CSE, National Institute of Technology Warangal, Warangal-506004, India*
[c]*Department of IT, Puducherry Technological University, Puducherry-605014, India*
uprabu28@gmail.com, venkatasai200206@gmail.com, satwikcse006@gmail.com, sanjaya@nitw.ac.in, vgeetha@ptuniv.edu.in

Abstract

Software-defined networking (SDN) is an emerging technology that addresses the limitations of legacy networks when handling the growing traffic demands. Legacy networks face issues like heavy traffic loads, congestion and delays. SDN offers a solution to these challenges. However, an efficient routing algorithm is required to transmit data packets through the network. While there are existing routing algorithms, there remains a need for a highly efficient one that minimizes packet loss and optimizes link utilization. One commonly used routing for mitigating issues, such as packet loss, delay and bandwidth allocation, is multipath routing. The inherent flexibility of SDN, which provides a comprehensive overview of network status, empowers routing algorithms to generate efficient routing paths. We introduce the enhanced dynamic multipath routing (EDMR) algorithm in this context. This algorithm dynamically generates and updates routing paths within the network's switches. Its dynamic nature significantly reduces packet loss and minimizes end-to-end delays.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Software-Defined Networking; Routing Algorithms; Multi-Path Routing; Network Load; Packet Loss; End-to-End Delay.

1. Introduction

In the contemporary world, an immense volume of data is generated and exchanged among various devices, resulting in a significant increase in network traffic [20]. This surge in data traffic has a detrimental effect on the network's performance and the quality of service (QoS) metrics, primarily due to inadequate management. Consequently, there is a demand for a more advanced networking paradigm to handle this substantial data load effectively. This new paradigm should exhibit flexibility, programmer-friendliness, and an ability to enhance network performance by optimizing QoS and maximizing the utilization of network resources. SDN emerges as a promising networking model that addresses these requirements [12]. SDN represents an advanced networking model characterized by the centralization

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.191

of network control. In this paradigm, a central controller manages all network controls and decisions. At the same time, the networking resources in the data plane, like switches, are primarily tasked with forwarding data packets from source to destination. Notably, SDN segregates the network into two distinct planes, the data and control planes [15]. To facilitate communication and coordination between these two planes, the southbound application programming interface (API) is employed. This API leverages the OpenFlow protocol, enabling the controller to instruct the switches on how to route data packets. The controller is the pivotal component in the SDN network and is responsible for programming all the decisions that need to be implemented on the network resources. The architecture of SDN is explained in [21].

Routing is key in networking [4, 23]. While single-path routing has been a common approach, solely transmitting data through the single best path often results in congestion and deteriorates network performance. To address these issues, multipath routing techniques have been introduced, which involve creating multiple paths for data transmission. This alleviates network device load and minimizes congestion [19]. However, it's important to note that multipath routing has its limitations. For instance, it may not be suitable for densely or lightly loaded networks [14]. Therefore, multipath routing can be categorized into per-packet and per-flow routing [13]. In the per-packet mechanism, data packets from the same flow traverse different paths, whereas the per-flow mechanism segregates packets from different flows into distinct paths. In the context of this paper, multipath routing is implemented using the depth-first search (DFS) algorithm, a graph-searching algorithm commonly employed in routing [8]. The proposed algorithm for multipath routing supports various network topologies like Abilene and Goodnet [1, 10].

The remaining sections are organized as follows. Section 2 discusses the related work. Section 3 presents the proposed algorithm in terms of topology construction, path generation and optimal path generation. Section 4 shows the experimental results of the Abilene and Goodnet topologies in terms of packet loss, average end-to-end delay and average round-trip time (RTT). Section 5 concludes the paper with some future work.

2. Related Work

Luo et al. [16] have proposed an adaptive dynamic multipath computation framework (ADMCP). It is a comprehensive platform designed to facilitate integrated resource management and provide a suite of routing and resource allocation management applications. It is adapted in centrally controlled and loosely coupled distributed SDN environments, particularly large-scale network systems. ADMCP comprises key components, such as SNOS-DW, flow pattern analysis, feature extraction for SDN-enabled network controllers, path optimization and more. The primary objective of this framework is to generate multiple (disjoint) paths offline and in real-time to efficiently manage routing and resource allocation in various network states and traffic conditions. Megyesi et al. [17] have proposed that available bandwidth is one of the best metrics for the cost function. The bandwidth estimators are presented using software probes and flow-based measurements. Software probes are software-based tools that can be installed on network devices to measure the available bandwidth. Flow-based measurements involve monitoring the flow of packets in the network to measure available bandwidth. Northbound API helps in monitoring bandwidth. This API provides bandwidth statistics to the estimators to calculate the available bandwidth. Accurate measurement of available bandwidth is essential for effective network management and ensuring a high-quality user experience. Alnaser et al. [2] have proposed a method for finding optimal routes using the branch and bound method. This method calculates the optimal paths by traversing level by level to reach the destination. A modified multistage graph algorithm finds the paths between source and destination. Hop count is used as a cost metric.

Jiawei et al. [9] have proposed a new priority-based dynamic routing algorithm. The algorithm allocates the resources according to the classes categorized based on the user's requirements. Therefore, the resources are efficiently utilized, which improves the QoS. The data packets that arrive at the controller are arranged in the queue, which helps to send the packets to the destination based on the queue's priority. Casas-Velasco et al. [5] have introduced an online QoS routing algorithm that utilizes reinforcement learning (RL) to optimize routing based on network conditions and QoS requirements. The RL agent considers network state and QoS criteria to select the best path by employing Q-learning. This approach includes a knowledge plane within SDN to learn network behaviour, enhancing path computation. The RL agent enriches the route data repository by identifying optimal routes for each source-destination pair. Compared to Dijkstra's algorithm, this method provides notable benefits. Awad et al. [3] have presented machine learning-based multipath routing (MLMR) as a deep neural network (DNN) based routing technique. The DNN model is trained in the application layer, and predicted solutions are transferred to the central controller. A heuristic, named column generation-based algorithm, generates the paths based on the network status. However, due to their high computational time, these routing solutions are used for initializing and training the DNN model. DNN model

with multiple hidden layers gets trained based on the real-time network status and heuristic routing solutions. The final model can predict the heuristic solution in less than 1% computation time. Syaifuddin et al. [22] have presented the modified DFS to get the shortest path to reach the destination. They have compared the modified DFS with DFS and Dijkstra's algorithms. The modified DFS algorithm finds all possible paths from source to destination and calculates the weight of each path to determine the shortest path. They have considered the throughput, jitter and packet loss as QoS parameters.

Elbasheer et al. [7] have proposed a dynamic QoS algorithm over SDN. This algorithm is designed to choose the most suitable path that aligns with video QoS requirements, ultimately enhancing users' quality of experience (QoE). The algorithm categorizes traffic according to video resolution. This routing strategy prioritizes fulfilling two critical routing criteria: minimizing packet loss and ensuring sufficient bandwidth availability. Zhang et al. [25] have presented a method for routing optimization in SDN considering multiple QoS constraints and a novel routing optimization algorithm by improving the traditional shuffled frog leaping (SFLA) algorithm, called the improved SFLA algorithm. The improved SFLA method introduces various capabilities, including encoding, subgroup segmentation and global search computations. Unlike the traditional SFLA, which groups data packets separately, this algorithm ensures that each packet efficiently identifies the optimal path from its source to the destination, thereby enhancing the routing optimization process. Chen et al. [6] have presented the RL-based multipath routing (RLMR) algorithm, which employs RL principles, Q-learning, and the Markov decision process (MDP) for training. RLMR focuses on reducing packet loss and jitter while optimizing link bandwidth utilization. It adjusts dynamically to network conditions, generating fresh routing paths when needed and selecting the most suitable path based on flow requirements. The model can create QoS routes and handle insufficient link bandwidth by dividing flows into sub-flows. The algorithm's effectiveness is compared to the datacenter using equal cost multipath (ECMP) and Dijkstra's shortest path algorithm for evaluation.

Zhang et al. [24] have introduced an online QoS-routing by considering deep reinforcement learning (DRL) and SDN, called DSOQR. This routing algorithm integrates asynchronous advantage actor-critic (A3C) and QoS routing. DSOQR enables switches to adapt paths dynamically based on current network conditions. A DRL agent selects optimal paths considering QoS criteria, such as latency, jitter and packet loss and updates its policy based on QoS feedback. This approach is compared to ECMP and K shortest paths (KSP) algorithms for evaluation. Kamboj et al. [11] have introduced a scheme focusing on splitting flows, routing through the network and reordering the flows at the destination. When splitting flows, the link's weight is taken into account. The weight of the link is calculated based on the link capacity and delay. The flows are distributed using a round-robin approach. The divided flows are routed through the network based on the flows selected by the k-nearest algorithm. At the destination, packets are reordered based on the sequence numbers. They have used AttMps and Goodnet topologies from Intenet Zoo to test their algorithm.

3. Proposed Algorithm

The architecture of the proposed algorithm is represented in Fig. 1. This figure illustrates the algorithm's structure and provides a comprehensive overview of the experiment's workflow, explaining the process of conducting the experiments and utilizing the obtained data. It specifies the key points where routing operations occur and how the resulting values are integrated into the experiment's framework. In essence, it offers an in-depth explanation of the entire experimental setup.

3.1. Data Collection

This paper utilizes real-time network topologies from the Internet zoo topology [18]. Two specific topologies, Abilene and Goodnet, are shown visually in Fig. 2 and Fig. 3, respectively. The Abilene topology comprises 12 nodes interconnected by 30 edges, while the Goodnet topology comprises 17 nodes linked by 31 edges. These selected topologies serve as the foundation for evaluating and assessing the performance of the proposed algorithm.

3.2. Topology Construction

The process of topology construction involves generating a network topology within the experimental setup that closely mimics real-world environments. To accomplish this, datasets like the Internet Zoo topology are used, which

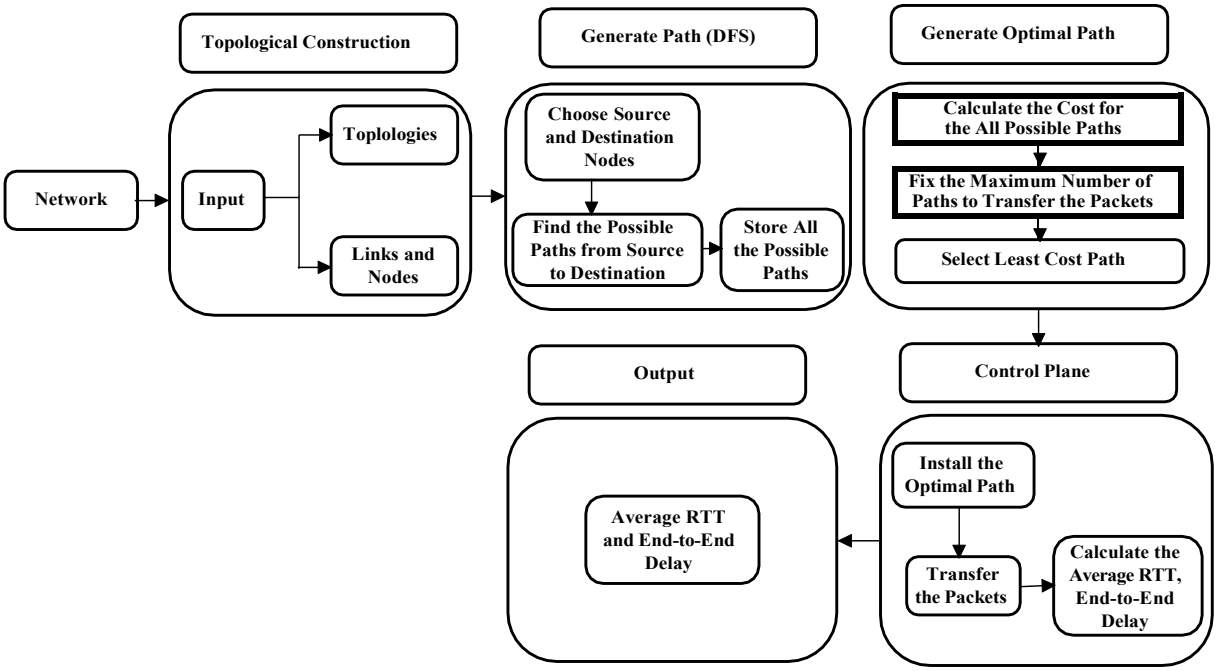


Fig. 1. The proposed architecture.

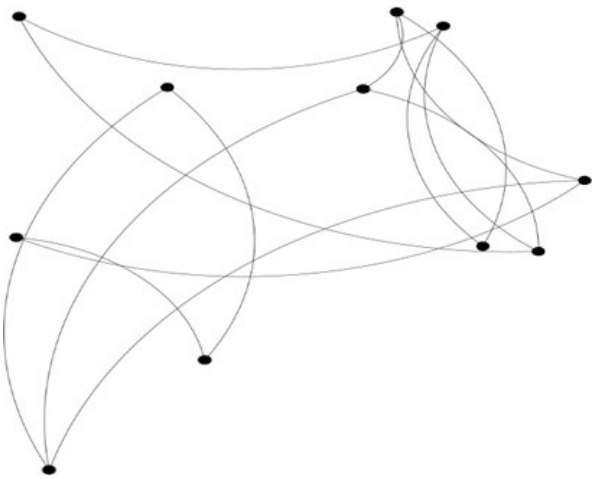


Fig. 2. Abilene topology.

include representations of topologies such as Abilene, Goodnet, and AttMps to support multipath routing. The Gephi tool is instrumental in visualizing these topologies. This tool requires the topology data to be in a comma-separated value (CSV) format, with the CSV file specifying the edges between nodes for the given topology. Once this CSV file is available, the topology can be manually built using a tool like MiniEdit. A Python script is then generated to represent the entire topology after incorporating switches, links, and a controller. The process of topology construction is outlined in Algorithm 1.

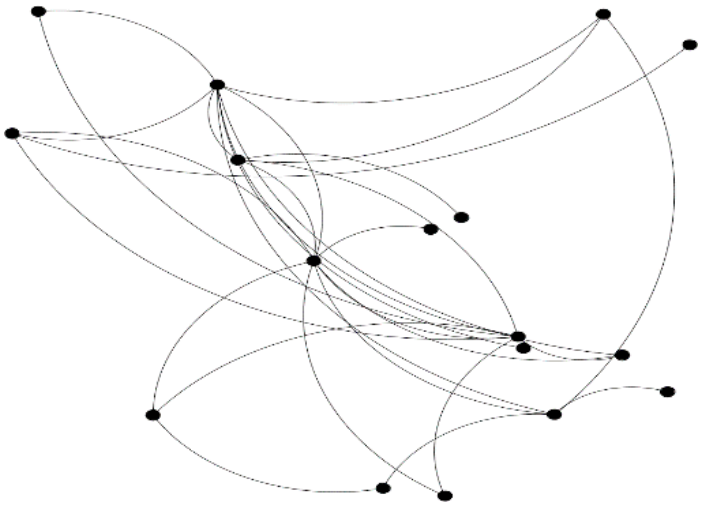


Fig. 3. Goodnet topology.

Algorithm 1 Topology Construction

1: Create a Mininet object with no topology, not building it yet, and an Internet protocol (IP) base of 10.0.0.0/8.
2: Add a RemoteController object with name ‘c0’, IP address ‘127.0.0.1’, protocol transmission control protocol (‘tcp’) and port 6633 to the Mininet object.
3: Add five Open vSwitch (OVS) KernelSwitch objects with names ‘s1’, ‘s2’, ‘s3’, ‘s4’ and ‘s5’, respectively, to the Mininet object.
4: Add two host objects with names ‘h1’ and ‘h2’, IP addresses ‘10.0.0.1’ and ‘10.0.0.2’ and no default route, respectively, to the Mininet object.
5: Add links between the hosts and switches to create a network topology. Build the network topology but do not start it yet.
6: Start the controllers and switches in the network.
7: Execute a custom-defined menu function that allows the user to interact with the network topology.
8: Generate the Python script file.

3.3. Path Generation

The network infrastructure is activated after starting the controller and activating all links and switches. Subsequently, the system generates all available paths based on the IP addresses of the source node and the destination node. This path-generation process involves systematically traversing nodes, level by level, to identify the destination. The resulting paths are stored in a list format, representing all the potential routes from the source to the destination. The step-by-step procedure for path generation is shown in Algorithm 2.

3.4. Optimal Path Generation

Constraints like available bandwidth, load, or delay can be used to find optimal paths. These parameters are considered based on the applications, and a cost function is produced. This cost function decides whether the path is suitable for transferring the packets. Optimal path generation uses bandwidth to calculate the path’s cost, and the path with the least cost is chosen as an optimal path. The cost function will select *k*-shortest paths from the available paths at most. The cost function is provided as follows.

$$c = \frac{REFERENCE_Bw}{b} \tag{1}$$

Algorithm 2 PathGeneration

```
    Inputs: sourceNode, destinationNode
    Output: All possible paths from source to destination
 1: if sourceNode == destinationNode then
 2:     return SourceNode
 3: end if
 4: Path=[] /*Initialize the stack with node and path*/
 5: stack = [(sourceNode, [sourceNode])]
 6: while stack do
 7:     node, path = pop the stack
 8:     for next in set(graphMatrix[node].keys() - set(path)) do
 9:         if next == destinationNode then
10:             path.append(path+[next])
11:         else
12:             stack.append((next, path+[next]))
13:         end if
14:     end for
15: end while
16: return path
```

where c is the link's cost, which indicates the effective bandwidth of the link, and *REFERENCE_Bw* is the constant value that is taken as a reference. The reference bandwidth is 10 Gbps, and b is the available bandwidth in that link. The link cost is calculated using Eq. 1. Cumulating each link cost gives the cost of the path. The k paths with the lowest costs are selected and installed in the switches for transferring the data packets. Here, k is chosen as 3. Algorithm 3 depicts the process of selecting the optimal paths.

Algorithm 3 Optimal Path Generation

```
    Inputs: SourceNode, DestinationNode
/*path generation generates all possible paths from source to destination*/
    Output: Optimal path
 1: paths = PathGeneration(SourceNode, DestinationNode)
 2: path_count = min(length(paths), max_paths) /*max_paths values vary with topology and parameters considered*/
 3: Path_costs = []
 4: for i in paths do
 5:     Path_costs.append(get_path_cost(i)) /*get_path_cost calculates the cost of the path*/
 6: end for
 7: Sorted_paths = []
 8: for all i belongs to path_counts do
 9:     min_index = -1
10:     min_cost = -1 * float("inf")
11:     for all j belongs to len(paths) do
12:         if path[j] < min_cost and paths[j] in Sorted_paths then
13:             min_index = j
14:             Min_cost = path_cost[j]
15:         end if
16:         Sorted_paths.append(paths[min_index])
17:     end for
18: end for
19: return Sorted_paths
```

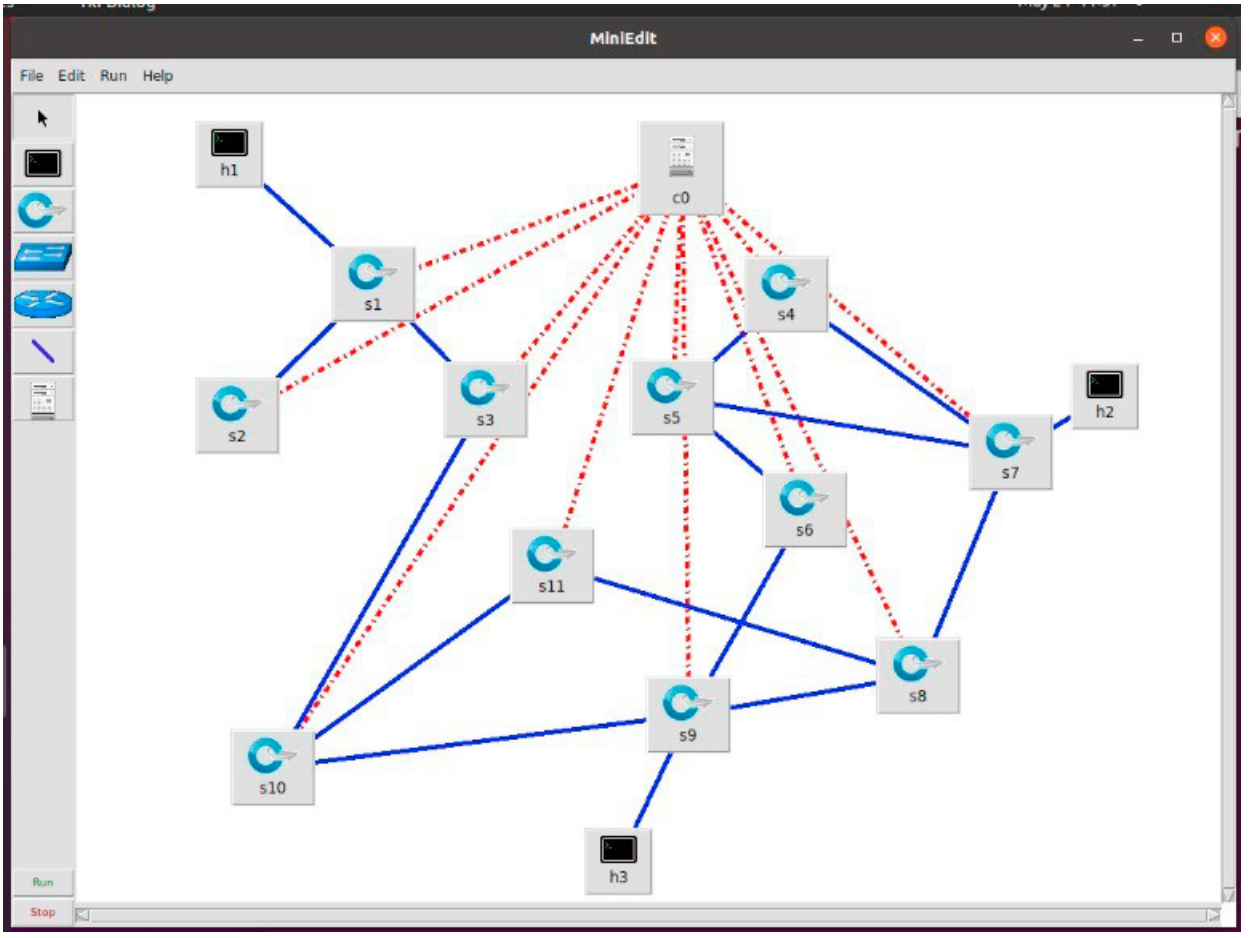


Fig. 4. MiniEdit.

4. Experimental Results

This section describes the outputs and the results obtained by the proposed algorithm. In the first step, topology needs to be constructed using the MiniEdit. For instance, the Abilene topology is constructed by dragging and dropping the controller and switches from the Gephi and adding suitable links between the switches, as shown in Fig. 4. Then, the file is saved as a Python script. After constructing the topology using the MiniEdit, it is required to start the controller through which the controller can analyze the network structure and links between the switches. It helps to manage the network and maintains the status of the whole network. After starting the controller, the topology needs to be initialized. It means that the topology should be able to accept and transfer the packets. As stated earlier, two different topologies, Abilene and Goodnet, are used for the experimentation.

The proposed algorithm's performance undergoes evaluation to assess its accuracy and effectiveness. The evaluation process measures key performance metrics: packet loss, end-to-end delay, and average RTT. Given the small-scale topology and the use of small-sized pings in our implementation, packet loss is negligible. Packet loss refers to the failure of one or more data packets to reach their intended destination in a network. Ideally, acceptable packet loss should remain below 5%. The packet loss, p, is the ratio of the difference between the total packets sent (p_s) and total packets received (p_r) to the p_s multiplied by 100. Mathematically,

$$p = \frac{p_s - p_r}{p_s \times 100} \tag{2}$$

```
Available paths from  1  to  7  :  [[1, 3, 10, 11, 8, 7], [1, 3, 10, 11, 8, 9, 6, 5, 7]
, [1, 3, 10, 11, 8, 9, 6, 5, 4, 7], [1, 3, 10, 9, 6, 5, 7], [1, 3, 10, 9, 6, 5, 4, 7],
[1, 3, 10, 9, 8, 7]]
[1, 3, 10, 11, 8, 7] cost =  5.0
[1, 3, 10, 9, 8, 7] cost =  5.0
[1, 3, 10, 9, 6, 5, 7] cost =  6.0
Path installation finished in  0.005345344543457031
Output port is 4 1684909188.9566262
```

Fig. 5. List of Paths.

```
Packet loss: 0.0%
Average end-to-end delay: 0.92 ms
Average RTT: 8.16 ms
```

Fig. 6. Results of three performance metrics.

End-to-end delay calculates the time a packet travels from source to destination within a network. This measurement relies on timestamps, specifically the time at which it is received (t_r) and the time at which it is sent (t_s). Mathematically,

$$d = (t_r - t_s) \times 100 \tag{3}$$

RTT signifies the duration required for a packet to travel from the source to the destination and then return. Typically, RTT is measured by transmitting a packet from the source and tracking the time it takes for the corresponding response packet to return. Mathematically,

$$RTT = t_2 - t_1 \tag{4}$$

where t_1 is the time when the packet is sent, and t_2 is the time when the corresponding response packet is received. The average RTT ($AvgRTT$) is calculated as follows.

$$AvgRTT = \frac{The\ sum\ of\ all\ RTT\ measurements}{Total\ number\ of\ measurements} \tag{5}$$

The path generation module generates all possible paths from source to destination. Using the cost function, it found the optimal paths from the generated path list.

Fig. 5 shows all the possible paths and the cost of the three least-cost paths. Finally, performance metrics are determined on the terminal, such as packet loss, average end-to-end delay and average RTT, as shown in Fig. 6. Table 1 and Table 2 show the values measured in the Abilene and Goodnet networks using the Ryu controller and SDN switches as network resources in the mininet emulator.

Table 1. Average RTT and end-to-end delay in the Abilene topology

Source	Destination	Paths	Cost	Ping1	Ping2	Ping3	Average RTT	Average End-to-End Delay
h1	h2	[1, 3, 10, 11, 8, 7]	5	0.34 ms	0.09 ms	0.24 ms	0.22 ms	0.92 ms
		[1, 3, 10, 9, 8, 7]	5					
		[1, 3, 10, 9, 6, 5, 7]	6					
h2	h3	[7, 8, 9]	2	0.07 ms	0.24 ms	0.17 ms	0.16 ms	0.92 ms
		[7, 5, 6, 9]	3					
		[7, 4, 5, 6, 9]	4					
h1	h3	[1, 3, 10, 9]	3	0.15 ms	0.22 ms	0.23 ms	0.20 ms	0.92 ms
		[1, 3, 10, 11, 8, 9]	5					
		[1, 3, 10, 11, 8, 7, 5, 6, 9]	8					

Table 2. Average RTT and end-to-end delay in the Goodnet topology

Source	Destination	Paths	Cost	Ping1	Ping2	Ping3	Average RTT	Average End-to-End Delay
h1	h2	[1, 4, 10, 2]	3	0.13 ms	0.20 ms	0.19 ms	0.17 ms	0.92 ms
		[1, 8, 16, 2]	3					
		[1, 8, 13, 16, 2]	4					
h2	h3	[2, 16, 17]	2	0.17 ms	0.28 ms	0.12 ms	0.19 ms	0.92 ms
		[2, 10, 16, 17]	3					
		[2, 10, 15, 16, 17]	4					
h1	h3	[1, 4, 16, 17]	3	0.15 ms	0.20 ms	0.15 ms	0.16 ms	0.92 ms
		[1, 8, 16, 17]	3					
		[1, 4, 10, 16, 17]	4					

5 Conclusion and Future Work

We introduced the EDMR algorithm to address the growing challenges posed by increasing network traffic, harnessing the capabilities of SDN. Our analysis of EDMR's performance included evaluating its impact on packet loss, average end-to-end delay, and average RTT metrics. These assessments were conducted on the Abilene and Goodnet network topologies. In the future, the focus will be to improve the throughput of EDMR and make it work for complex real-time networks. Additionally, the focus will be to enhance the QoS parameters in the network.

References

[1] AlGhadhban, A., Shihada, B., 2016. Energy efficient sdn commodity switch based practical flow forwarding method, in: NOMS 2016-2016 IEEE/IFIP Network Operations and Management Symposium, IEEE. pp. 784–788.

[2] Alnaser, M., 2018. A method of multipath routing in sdn networks. Advances in Computer Science and Engineering 17, 11–17.

[3] Awad, M.K., Ahmed, M.H.H., Almutairi, A.F., Ahmad, I., 2021. Machine learning-based multipath routing for software defined networks. Journal of Network and Systems Management 29, 1–30.

[4] Bhoi, S.K., Puthal, D., Khilar, P.M., Rodrigues, J.J., Panda, S.K., Yang, L.T., 2018. Adaptive routing protocol for urban vehicular networks to support sellers and buyers on wheels. Computer Networks 142, 168–178.

[5] Casas-Velasco, D.M., Rendon, O.M.C., da Fonseca, N.L., 2020. Intelligent routing based on reinforcement learning for software-defined networking. IEEE Transactions on Network and Service Management 18, 870–881.

[6] Chen, C., Xue, F., Lu, Z., Tang, Z., Li, C., et al., 2022. Rlmr: Reinforcement learning based multipath routing for sdn. Wireless Communications and Mobile Computing 2022.

[7] Elbasheer, M.O., Aldegheishem, A., Lloret, J., Alrajeh, N., 2021. A qos-based routing algorithm over software defined networks. Journal of Network and Computer Applications 194, 103215.

[8] Jiang, J.R., Huang, H.W., Liao, J.H., Chen, S.Y., 2014. Extending dijkstra's shortest path algorithm for software defined networking, in: The 16th Asia-Pacific Network Operations and Management Symposium, IEEE. pp. 1–4.

[9] Jiawei, W., Xiuquan, Q., Chen, J., 2019. Pdmr: priority-based dynamic multi-path routing algorithm for a software defined network. IET Communications 13, 179–185.

[10] Jo, E., Pan, D., Liu, J., Butler, L., 2014. A simulation and emulation study of sdn-based multipath routing for fat-tree data center networks, in: Proceedings of the Winter Simulation Conference 2014, IEEE. pp. 3072–3083.
[11] Kamboj, P., Pal, S., Bera, S., Misra, S., 2022. Qos-aware multipath routing in software-defined networks. IEEE Transactions on Network Science and Engineering 10, 723–732.
[12] Kreutz, D., Ramos, F.M., Verissimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S., 2014. Software-defined networking: A comprehensive survey. Proceedings of the IEEE 103, 14–76.
[13] Li, M., Lukyanenko, A., Ou, Z., Ylä-Jääski, A., Tarkoma, S., Coudron, M., Secci, S., 2016. Multipath transmission for the internet: A survey. IEEE Communications Surveys & Tutorials 18, 2887–2925.
[14] Liu, X., Mohanraj, S., Pioro, M., Medhi, D., 2014. Multipath routing from a traffic engineering perspective: How beneficial is it?, in: 2014 IEEE 22nd International Conference on Network Protocols, IEEE. pp. 143–154.
[15] Lo, C.C., Wu, P.Y., Kuo, Y.H., 2015. Flow entry conflict detection scheme for software-defined network, in: 2015 International Telecommunication Networks and Applications Conference (ITNAC), IEEE. pp. 220–225.
[16] Luo, M., Zeng, Y., Li, J., Chou, W., 2015. An adaptive multi-path computation framework for centrally controlled networks. Computer Networks 83, 30–44.
[17] Megyesi, P., Botta, A., Aceto, G., Pescape´, A., Molna´r, S., 2017. Challenges and solution for measuring available bandwidth in software defined networks. Computer Communications 99, 48–61.
[18] Nguyen, H., Falkner, N., Roughan, M., . The internet topology zoo. http://www.topology-zoo.org/dataset.html. Accessed: 2023-09-10.
[19] Ramdhani, M.F., Hertiana, S.N., Dirgantara, B., 2016. Multipath routing with load balancing and admission control in software-defined networking (sdn), in: 2016 4th International Conference on Information and Communication Technology (ICoICT), IEEE. pp. 1–6.
[20] Sahoo, K.S., Panda, S.K., Sahoo, S., Sahoo, B., Dash, R., 2019. Toward secure software-defined networks against distributed denial of service attack. The Journal of Supercomputing 75, 4829–4874.
[21] Shirmarz, A., Ghaffari, A., 2020. An adaptive greedy flow routing algorithm for performance improvement in software-defined network. International Journal of Numerical Modelling: Electronic Networks, Devices and Fields 33, e2676.
[22] Syaifuddin, S., Azis, M.F., Sumadi, F.D.S., 2021. Comparison analysis of multipath routing implementation in software defined network. Kinetik: Game Technology, Information System, Computer Network, Computing, Electronics, and Control .
[23] Thanedar, M.A., Panda, S.K., 2023. A dynamic resource management algorithm for maximizing service capability in fog-empowered vehicular ad-hoc networks. Peer-to-Peer Networking and Applications , 1–15.
[24] Zhang, L., Lu, Y., Zhang, D., Cheng, H., Dong, P., et al., 2022. Dsoqr: Deep reinforcement learning for online qos routing in sdn-based networks. Security and Communication Networks 2022.
[25] Zhang, Y., Gorlatch, S., 2021. Optimizing energy efficiency of qos-based routing in software-defined networks, in: Proceedings of the 17th ACM Symposium on QoS and Security for Wireless and Mobile Networks, pp. 87–94.